@aman_views

MAVEN HANDBOOK

1. WHAT IS MAVEN?

Maven is a build automation and project management tool used mainly for Java projects.
It simplifies the build process, dependency management and project management.

2. WHY MAVEN?

- ★ Automates the build process
- ★ Manages dependencies
- ★ Standard project structure
- ★ Supports plugins
- ★ Easy integration with IDEs & CI/CD
- ★ Reproducible builds

3. MAVEN PROJECT STRUCTURE

```
MyProject/
├── src/
│   ├── main/
│   │   ├── java/        → Java source files
│   │   └── resources/   → Resource files
│   └── test/
│       ├── java/        → Test source files
│       └── resources/   → Test resources
├── pom.xml              → Project configuration
└── target/              → Compiled output
```

4. POM.XML – BASIC STRUCTURE

```xml
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-project</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>
  <dependencies>
    <!-- Add dependencies here -->
  </dependencies>
</project>
```

groupId	: Unique ID of your organization (reverse domain name)
artifactId	: Unique name of the project
version	: Version of the project
packaging	: Type of package (jar, war, etc.)
dependencies	: List of libraries required for the project

5. MAVEN BUILD LIFECYCLE

- **validate** → Validates the project and checks if everything is correct.
- **compile** → Compiles the source code.
- **test** → Runs unit tests using a test framework.
- **package** → Creates the package (JAR/WAR).
- **verify** → Runs any checks on the package.
- **install** → Installs the package in the local repository.
- **deploy** → Deploys the package to the remote repository.

☆ NOTE

Maven follows convention over configuration.
If you follow the standard structure and naming, Maven will handle the rest!

@aman_views

MAVEN HANDBOOK

1. POM.XML - HEART OF MAVEN

- pom.xml (Project Object Model) is the configuration file of a Maven project.
- It contains project information, dependencies, plugins and other build settings.

2. POM.XML STRUCTURE

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                  http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>my-project</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <name>My Project</name>
  <description>Sample Maven Project</description>

  <properties>
     <java.version>17</java.version>
  </properties>

  <dependencies>
     <!-- Add project dependencies here -->
  </dependencies>

  <build>
     <!-- Build configurations & plugins -->
  </build>

</project>
```

→ **modelVersion :**
Version of the POM model.

→ **groupId :**
Unique ID of your organization (reverse domain name).

→ **artifactId :**
Unique name of the project.

→ **version :**
Version of the project.

→ **packaging :**
Type of package (jar, war, pom, etc.)
Default is jar.

→ **name :**
Name of the project.

→ **description :**
Short description of the project.

→ **properties :**
Define reusable values.
Helps in maintainability.

→ **dependencies :**
List of libraries required by the project.

→ **build :**
Contains plugins and build specific configurations.

3. DEPENDENCY DECLARATION - SYNTAX

```xml
<dependency>
   <groupId>org.testng</groupId>
   <artifactId>testng</artifactId>
   <version>7.8.0</version>
   <scope>test</scope>   <!-- optional -->
</dependency>
```

- **groupId** : Organization or group.
- **artifactId** : Name of the library.
- **version** : Version of the library.
- **scope** : Defines where this dependency is used.
  If scope not mentioned, default is **compile**.

4. COMMON DEPENDENCY SCOPES

Scope	Description	Available in
compile (default)	Available in all phases. Required for compile, test and runtime.	Compile + Test + Runtime
test	Only for test phase. Not included in the final package.	Test
provided	Provided by JDK or container at runtime.	Compile + Test
runtime	Needed only at runtime. Not required for compile.	Runtime
system	Similar to provided. You must specify systemPath.	Compile + Test
import	Used in dependencyManagement to import other POMs.	Compile

☆ **TIPS :** Always use latest stable versions, avoid SNAPSHOT in production and keep pom.xml clean and organized.

MAVEN HANDBOOK

@aman_views

1. MOST USED MAVEN COMMANDS

Category	Command	Description
BUILD	mvn compile	Compiles the source code.
	mvn test	Runs unit tests using the test framework.
	mvn package	Packages the compiled code into JAR/WAR.
	mvn verify	Runs any checks to verify the package is valid.
CLEANING	mvn clean	Deletes the target directory.
	mvn clean install	Cleans and installs the package in local repository.
INSTALL & DEPLOY	mvn install	Installs the package in local repository (~/.m2/repository).
	mvn deploy	Deploys the package to remote repository.
INFO & UTILITY	mvn validate	Validates the project and checks if everything is correct.
	mvn help	Displays help information.
	mvn dependency:tree	Shows dependency tree of the project.
	mvn versions:display-dependency-updates	Checks available dependency updates.

2. MAVEN PROFILES

- Profiles allow you to provide different configurations for different environments.
- Activated using a specific id.

```
<profiles>
  <profile>
    <id>dev</id>
    <properties>
      <env>dev</env>
    </properties>
  </profile>
</profiles>
```

Activate Profile :

```
mvn clean install -Pdev
```

3. MAVEN PROPERTIES

- Properties store values that can be reused in pom.xml.
- Defined inside <properties> tag.

```
<properties>
  <java.version>17</java.version>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

Use in POM :

```
<version>${java.version}</version>
```

Built-in Properties Example :

- ${project.artifactId}
- ${project.version}
- ${project.build.directory}
- ${basedir}
- ${user.home}
- ${java.version}

4. MAVEN SETTINGS.XML (USER LEVEL CONFIGURATION)

- Located at : ~/.m2/settings.xml
- Used to define repositories, credentials, mirrors, proxies, and profiles.
- Example (Repository configuration) :

```
<settings>
  <profiles>
    <profile>
      <id>my-repo</id>
      <repositories>
        <repository>
          <id>my-repo</id>
          <url>https://repo.example.com/maven2</url>
        </repository>
      </repositories>
    </profile>
  </profiles>
</settings>
```

5. .M2 REPOSITORY

- Maven stores all downloaded dependencies, plugins and metadata in your local repository.
- Default location : ~/.m2/repository
- Helps in faster builds as dependencies are re-used.

Folder Structure Example :

```
~/.m2/repository/
└ org
  └ testng
    └ testng
      └ 7.8.0
        ├ testng-7.8.0.jar
        ├ testng-7.8.0.pom
        └ _remote.repositories
```

NOTE : Maven follows convention over configuration. Standard structure and naming helps Maven to handle the rest!

MAVEN HANDBOOK

@aman_views

1. IMPORTANT MAVEN PLUGINS

Plugin	Purpose
maven-compiler-plugin	Compiles the source code. Allows setting Java version.
maven-surefire-plugin	Runs unit tests (JUnit, TestNG). Generates test reports.
maven-failsafe-plugin	Runs integration tests. Used in verify phase.
maven-jar-plugin	Creates JAR files.
maven-war-plugin	Creates WAR files.
maven-shade-plugin	Creates an uber/fat JAR with all dependencies.
maven-source-plugin	Attaches source code (sources.jar).
maven-javadoc-plugin	Generates Javadoc documentation.

2. POM INHERITANCE & PARENT POM

- Parent POM helps to manage common configuration for multiple modules.
- Child modules inherit configuration from parent.

```
<parent>
    <groupId>com.example</groupId>
    <artifactId>parent-project</artifactId>
    <version>1.0.0</version>
</parent>
```

3. MULTI-MODULE PROJECT

- Used to break large project into smaller modules.
- Managed from a single parent POM.

```
<modules>
    <module>module-a</module>
    <module>module-b</module>
    <module>module-c</module>
</modules>
```

4. MAVEN vs GRADLE (QUICK COMPARISON)

Feature	Maven	Gradle
Build File	pom.xml (XML)	build.gradle (Groovy/Kotlin)
Convention	More Convention	More Configuration
Performance	Slower	Faster (Incremental Build)
Learning Curve	Easy	Moderate
Dependency Mgmt.	Excellent	Excellent
Best For	Standard Projects	Large / Complex Projects

5. TOP INTERVIEW QUESTIONS

1. **What is Maven?**
   - Build automation and project management tool.
2. **What is pom.xml?**
   - Project Object Model file. Contains all project configuration.
3. **What are the phases in Maven lifecycle?**
   - validate, compile, test, package, verify, install, deploy.
4. **What is the difference between install and deploy?**
   - install: Installs the package to local repository.
   - deploy: Deploys the package to remote repository.
5. **What are dependency scopes?**
   - compile, test, provided, runtime, system, import.
6. **What is the local repository in Maven?**
   - Exists at ~/.m2/repository. Stores all downloaded artifacts.
7. **What is the use of -U option?**
   - Forces Maven to update snapshots and releases.
8. **What is the use of clean phase?**
   - Deletes the target directory.
9. **What is a plugin in Maven?**
   - Plugins are used to perform specific tasks in the build lifecycle.
10. **How does Maven manage dependencies?**
    - It downloads dependencies from repositories and resolves transitive dependencies automatically.

6. MAVEN CHEAT SHEET (QUICK REFERENCE)

Task	Command
Compile source	mvn compile
Run tests	mvn test
Package project	mvn package
Install to local repo	mvn install
Deploy to remote repo	mvn deploy
Clean target directory	mvn clean
Clean and install	mvn clean install
Show dependency tree	mvn dependency:tree
Force update	mvn -U clean install
Help	mvn help

7. MAVEN FLOW (QUICK OVERVIEW)

pom.xml (Config) → Dependencies (Download) → Plugins (Execute) → Lifecycle (Phases) → Build (Output)

KEY TAKEAWAY

- Maven follows convention over configuration.
- Standard lifecycle, project structure and dependency management make it reliable and consistent.
- Write once, build anywhere!